

SWT 22022: Practical for Internet Application Development
Department of Information & Communication Technology
Faculty of Technology
South Eastern University of Sri Lanka

Lab Sheet No: 03

Date: - 2025.09.18

Title: Introduction to PHP Form and OOP concept.

Aims:

- Getting familiar with PHP Forms and OOP concept.

Tasks:

- PHP form validation
- Class declaration in PHP
- Define objects in PHP

Activities

1. PHP Form Validation

- **PHP stripslashes() Function**

The stripslashes() function removes backslashes. This function can be used to clean up data retrieved from a database or from an HTML form.

- **PHP trim() Function**

The trim() function removes whitespace and other predefined characters from both sides of a string.

Related functions:

ltrim() - Removes whitespace or other predefined characters from the left side of a string

rtrim() - Removes whitespace or other predefined characters from the right side of a string

- **`$_SERVER["PHP_SELF"]` variable**

The `$_SERVER["PHP_SELF"]` is a super global variable that returns the filename of the currently executing script.

- **PHP `htmlspecialchars()` function**

The `htmlspecialchars()` function converts special characters to HTML entities. This means that it will replace HTML characters like `<` and `>` with `<` and `>`. This prevents attackers from exploiting the code by injecting HTML or Javascript code (Cross-site Scripting attacks) in forms.

```
<!DOCTYPE HTML>
<html>
<head>
</head>

<body>

<?php
// define variables and set to empty values
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = test_input($_POST["name"]);
    $email = test_input($_POST["email"]);
    $website = test_input($_POST["website"]);
    $comment = test_input($_POST["comment"]);
    $gender = test_input($_POST["gender"]);
}

function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
} ?>
```

```
<h2>PHP Form Validation Example</h2>
```

```
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

```
Name: <input type="text" name="name"> <br><br>
```

```
E-mail: <input type="text" name="email"> <br><br>
```

```
Website: <input type="text" name="website"> <br><br>
```

```
Comment: <textarea name="comment" rows="5" cols="40"></textarea> <br><br>
```

```
Gender: <input type="radio" name="gender" value="female">Female
```

```
<input type="radio" name="gender" value="male">Male
```

```
<input type="radio" name="gender" value="other">Other
```

```
<br><br>
```

```
<input type="submit" name="submit" value="Submit">
```

```
</form>
```

```
<?php
```

```
echo "<h2>Your Input:</h2>";
```

```
echo $name; echo "<br>";
```

```
echo $email; echo "<br>";
```

```
echo $website; echo "<br>";
```

```
echo $comment;
```

```
echo "<br>";
```

```
echo $gender;
```

```
?>
```

```
</body>
```

```
</html>
```

- **PHP Form Required**

From the validation rules table on the previous page, we see that the "Name", "E-mail", and "Gender" fields are required. These fields cannot be empty and must be filled out in the HTML form.

Field Validation Rules

- Name Required. + Must only contain letters and whitespace
- E-mail Required. + Must contain a valid email address (with @ and .)
- Website Optional. If present, it must contain a valid URL
- Comment Optional. Multi-line input field (textarea)
- Gender Required. Must select one

```
<!DOCTYPE HTML>
<html>
<head>
<style>.error {color: #FF0000;} </style>
</head>
<body>
<?php // define variables and set to empty values
$nameErr = $emailErr = $genderErr = $websiteErr = ""; $name = $email = $gender = $comment =
$website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    if (empty($_POST["name"])) {
        $nameErr = "Name is required";
    }
    else {
        $name = test_input($_POST["name"]);
    }

    if (empty($_POST["email"])) {
        $emailErr = "Email is required";
    }
    else {
        $email = test_input($_POST["email"]);
    }
}
```

```

if (empty($_POST["website"])) {
    $website = "";
}
else {
    $website = test_input($_POST["website"]);
}

if (empty($_POST["comment"])) {
    $comment = "";
}
else {
    $comment = test_input($_POST["comment"]);
}

if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
}
else {
    $gender = test_input($_POST["gender"]);
}
}

function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}
?>

```

<h2>PHP Form Validation Example</h2>

<p>* required field</p>

<form method="post" action="<?php echo htmlspecialchars(\$_SERVER["PHP_SELF"]);?>">

Name: <input type="text" name="name"> * <?php echo \$nameErr;?>

E-mail: <input type="text" name="email"> * <?php echo \$emailErr;?>


```

Website: <input type="text" name="website"> <span class="error"><?php echo
$websiteErr;?></span> <br><br>
Comment: <textarea name="comment" rows="5" cols="40"></textarea> <br><br>
Gender: <input type="radio" name="gender" value="female">Female
<input type="radio" name="gender" value="male">Male
<input type="radio" name="gender" value="other">Other <span class="error">* <?php echo
$genderErr;?></span> <br><br>
<input type="submit" name="submit" value="Submit">
</form>

```

```

<?php echo "<h2>Your Input:</h2>";
echo $name; echo "<br>";
echo $email; echo "<br>";
echo $website; echo "<br>";
echo $comment; echo "<br>";
echo $gender; ?>
</body>
</html>

```

- **PHP Forms - Validate E-mail and URL**

- **PHP - Validate Name**

The code below shows a simple way to check if the name field only contains letters, dashes, apostrophes and whitespaces. If the value of the name field is not valid, then store an error message:

```

$name = test_input($_POST["name"]);
if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {
    $nameErr = "Only letters and white space allowed";
}

```

Note: The `preg_match()` function searches a string for pattern, returning true if the pattern exists, and false otherwise.

- **PHP - Validate E-mail**

```

$email = test_input($_POST["email"]);
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $emailErr = "Invalid email format";
}

```

- **PHP - Validate URL**

```
$website = test_input($_POST["website"]);
if (!preg_match("/^b(?:(:https?|ftp):\\V|www\\.)[-a-z0-9+&@#\\/%?=_~!|:,;]*[-a-z0-9+&@#\\/%=_~!|,;:]/i",$website)) {
    $websiteErr = "Invalid URL";
}
```

Full Script

```
<html>
<head>
<style>.error {color: #FF0000;} </style>
</head>

<body>

<?php // define variables and set to empty values
$nameErr = $emailErr = $genderErr = $websiteErr = "";
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    if (empty($_POST["name"])) {
        $nameErr = "Name is required";
    }
    else {
        $name = test_input($_POST["name"]); // check if name only contains letters and whitespace
        if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {
            $nameErr = "Only letters and white space allowed";
        }
    }
}

if (empty($_POST["email"])) {
    $emailErr = "Email is required"; }
else {
    $email = test_input($_POST["email"]); // check if e-mail address is well-formed
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $emailErr = "Invalid email format";
    }
}
}
```

```

if (empty($_POST["website"])) {
    $website = ""; }
else {
    $website = test_input($_POST["website"]);
    // check if URL address syntax is valid
    if (!preg_match("/^b(?:(:https?|ftp):\\V\\V|www\\.)[-a-z0-9+&@#\\V%?=_!\\.:,;]*[-a-z0-9+&@#\\V%=_~_]/i",$website)) {
        $websiteErr = "Invalid URL";
    }
}

```

```

if (empty($_POST["comment"])) {
    $comment = ""; }
else {
    $comment = test_input($_POST["comment"]);
}

```

```

if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
}
else {
    $gender = test_input($_POST["gender"]);
}
} // close of $_SERVER["REQUEST_METHOD"] == "POST"

```

```

function test_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}
?>

```

<h2>PHP Form Validation Example</h2>

<p>* required field</p>

<form method="post" action="<?php echo htmlspecialchars(\$_SERVER["PHP_SELF"]);?>">

Name: <input type="text" name="name"> * <?php echo \$nameErr;?>


```
E-mail: <input type="text" name="email"> <span class="error">* <?php echo $emailErr;?></span>
<br><br>
Website: <input type="text" name="website"> <span class="error"><?php echo
$websiteErr;?></span> <br><br>
Comment: <textarea name="comment" rows="5" cols="40"></textarea> <br><br>
Gender:
<input type="radio" name="gender" value="female">Female
<input type="radio" name="gender" value="male">Male
<input type="radio" name="gender" value="other">Other <span class="error">* <?php echo
$genderErr;?></span> <br><br>
<input type="submit" name="submit" value="Submit">
</form>

<?php
echo "<h2>Your Input:</h2>";
echo $name; echo "<br>";
echo $email; echo "<br>";
echo $website; echo "<br>";
echo $comment; echo "<br>";
echo $gender;
?>
</body>
</html>
```

2. Class declaration in PHP

Create an Animal class in PHP

```
<?php
    class Animal{
        public $family;
        public $food;

        function set_family($family){
            $this->family = $family;
        }

        function get_family(){
            return $this-> family;
        }
    }
?>
```

3. Define objects in PHP

Define objects in class using new keyword

```
<?php
// Define a class called "Person"
class Person {
    public $name;
    public $age;

    public function __construct($name, $age) {
        $this->name = $name;
        $this->age = $age;
    }

    public function greet() {
        echo "Hello, my name is " . $this->name . " and I am " . $this->age . " years old.<br>";
    }
}

// Create objects (instances) of the Person class
$person1 = new Person("John Doe", 30);
$person2 = new Person("Jane Smith", 25);

// Access object properties and call methods
echo "Person 1 Info:<br>";
$person1->greet();

echo "<br>Person 2 Info:<br>";
$person2->greet();

?>
```

Exercise

a. Create a class name Operations.

b. Inside operation class creates following functions

- Add function = need to insert 2 numbers and get summation of those 2 numbers.
- Sub function = need to insert 2 numbers and get subscription of those 2 numbers. (like number1 – number 2)
- Multi function = need to insert 2 numbers and get multification of those 2 numbers.
- Divide function = need to insert 2 numbers and get divided value of those 2 numbers.